

CS 162 Homework Companion Guide

=====

Fallback PDF generated by build-pdfs.js because browser PDF printing was unavailable.

CS 162 Homework Companion Guide

=====

Generated from the repository README files.

This guide is an original, repository-specific companion to the public CS 162 homework pages. It does not copy the official pages verbatim; instead, it maps the official task structure to this repository, adds build and run commands, and includes detailed testing guidance.

Repository Overview

=====

Source file: README.md

CS 162 Student Repository

=====

This repository contains code for the CS 162 individual homework assignments. The documentation in each homework directory is written in an assignment-report style and aligned with the public CS 162 homework pages.

Documentation index

- hw-intro <hw-intro/README.md>: HW 0, C tools, user limits, memory layout, GDB basics, and word counting.
- hw-list <hw-list/README.md>: HW 1, Pintos lists, pthread practice, and threaded word counting.
- hw-shell <hw-shell/README.md>: HW 2, directory commands, program execution, path resolution, redirection, pipes, and signal handling.
- hw-http <hw-http/README.md>: HW 3 C HTTP server, socket setup, GET handling, proxying, server variants, and performance.
- hw-http-rs <hw-http-rs/README.md>: HW 3 Rust HTTP server, socket handling, GET parsing, directory responses, and statistics.
- hw-memory <hw-memory/README.md>: HW 4, Pintos sbrk, user heap growth, and user-space allocation library functions.
- hw-map-reduce <hw-map-reduce/README.md>: HW 5 C MapReduce coordinator, worker, client, task scheduling, and fault tolerance.
- hw-map-reduce-rs <hw-map-reduce-rs/README.md>: HW 5 Rust MapReduce coordinator, worker registration, task distribution, and fault tolerance.

Enhanced guides and validation reports

Additional English documentation lives in
docs/cs162-homework-guides <docs/cs162-homework-guides/README.md>:

- A generated companion guide that combines the per-homework READMEs into one printable document.
- A test-completeness audit that records what was attempted locally, what was

blocked by the Windows environment, what static evidence was found, and what must be added before each homework can be considered well tested.

- A PDF generation script for rebuilding the printable artifacts after the README files change.

Source alignment

The homework summaries and test expectations are based on the current public CS 162 pages:

- <<https://cs162.org/static/hw/hw-intro/>>
- <<https://cs162.org/static/hw/hw-list/>>
- <<https://cs162.org/static/hw/hw-shell/>>
- <<https://cs162.org/static/hw/hw-http/>>
- <<https://cs162.org/static/hw/hw-http-rs/>>
- <<https://cs162.org/static/hw/hw-memory/>>
- <<https://cs162.org/static/hw/hw-map-reduce/>>
- <<https://cs162.org/static/hw/hw-map-reduce-rs/>>

Repository-wide test audit

This repository does not currently have complete local automated testing for every homework. The audit below describes the state of the repository, not the official course autograder. The local Windows environment used for this audit did not have make, cargo, or a configured WSL Linux distribution, so tests were not executed end-to-end here. Run the listed commands in Linux, WSL, or the CS 162 course environment.

Current coverage:

- hw-memory/pintos/src/tests/memory/Make.tests is the strongest test suite in this repository. It enables 26 active Pintos memory tests covering sbrk, malloc, free, and realloc behavior. calloc still deserves explicit additional coverage.
- hw-http-rs/src/tests/ has helper-level Rust tests for argument defaults, MIME type helpers, response helpers, and statistics.
- hw-shell/test.sh is a small smoke test for one built-in command and one external command.
- hw-memory/mm_alloc/mm_test.c is a minimal allocator smoke test.
- hw-map-reduce-rs/src/tests/mod.rs contains an ignored test_wc with a placeholder input path, so default cargo test does not validate a full MapReduce run.

Major missing coverage:

- hw-intro, hw-list, hw-http, and hw-map-reduce need checked-in test harnesses with deterministic fixtures and golden outputs.
- hw-shell needs tests for the full CS 162 shell surface: cd, pwd, external program execution, path resolution, redirection, pipes, Ctrl-D, Ctrl-C, and Ctrl-Z behavior.
- hw-http and hw-http-rs need real server integration tests that send HTTP requests over sockets and assert status lines, headers, bodies, directory responses, malformed requests, and concurrent clients.
- hw-map-reduce and hw-map-reduce-rs need integration tests that start a coordinator, start workers, submit jobs, compare output to golden files, and validate retry behavior after worker failure.

Each homework README gives detailed prompts for the exact tests that should be added next.

HW 0 Intro

=====

Source file: hw-intro/README.md

HW 0: Intro

=====

Official reference:

- <<https://cs162.org/static/hw/hw-intro/>>
- <<https://cs162.org/static/hw/hw-intro/docs/counting-words/>>

Assignment goals

This homework is the CS 162 warm-up for C programming and Unix development. It exercises compilation with make, command-line tools, resource limits, memory layout, debugging, and a small word-count program.

The word-count portion follows the official hw-intro/words task shape: implement the counting helpers, free all allocated structures, and make the main program handle one or more input files and command-line output modes.

Key implementation areas

- limits.c: inspect and print process resource limits with real values.
- map.c and recurse.c: observe process memory layout, stack growth, heap allocation, and recursive stack behavior.
- words/main.c: parse flags, open input streams, combine file results, and choose output format.
- words/word_count.c: implement num_words, count_words, and free_words.
- words/gutenberg/: larger text inputs for manual stress checks.

Build and run

```
cd hw-intro
make
./limits
./map
```

```
cd words
make
./words -h
./words -c words.txt
./words -f words.txt
./words -c gutenber/alice.txt
```

Current test inventory

No checked-in automated test harness was found for this homework. The current coverage is manual smoke testing only:

- Compile limits, map, and words.
- Run limits and inspect whether the output is plausible.
- Run map and inspect whether stack, heap, globals, and code addresses are distinguishable.
- Run words on sample files and compare output by hand.

Completeness assessment

Coverage is incomplete. The official homework expects several behaviors that should be asserted automatically:

- words should count words accurately for file input.
- Frequency output should be deterministic enough to compare against expected content after sorting if the program order is not specified.
- Multiple input files should be processed as one combined stream.
- Stdin behavior should work when no file path is supplied or when the chosen mode expects stream input.
- Empty files, missing files, invalid flags, punctuation, repeated words, and case behavior should be tested.
- limits and map need shape checks so placeholder or all-zero output is caught early.

Detailed missing-test prompts

1. Add words/tests/basic.txt:

```
""text
hello hello world
systems world hello
""
```

Add expected checks:

- ./words -c tests/basic.txt should report 6 total words.
- ./words -f tests/basic.txt should include hello with count 3, world with count 2, and systems with count 1.
- If output order can vary, normalize with sort before comparing.

2. Add words/tests/punctuation.txt:

```
""text
Hello, hello! HELLO?
cs-162 cs162
""
```

The test should document the expected tokenizer policy. If punctuation is a separator and case is preserved, check those exact tokens. If case is folded, check the folded counts. This fixture prevents silent changes to token parsing.

3. Add words/tests/multifile_a.txt and words/tests/multifile_b.txt:

```
""text
alpha beta
""
```

```
""text
beta gamma gamma
""
```

Test command:

```
""sh
./words -f tests/multifile_a.txt tests/multifile_b.txt
""
```

Expected content: alpha: 1, beta: 2, gamma: 2.

4. Add stdin coverage:

```
""sh
printf "one two one\n" | ./words -f
""
```

Assert one: 2 and two: 1. This catches programs that only work when fopen succeeds on a path.

5. Add failure-mode coverage:

```
""sh
./words -z tests/basic.txt
./words -c tests/does-not-exist.txt
""
```

Assert a non-zero exit status and a useful error message on stderr.

6. Add limits sanity coverage in hw-intro/test_intro.sh:

- Run ./limits.
- Fail if the output is empty.
- Fail if every numeric field is 0.
- Fail if expected limit names such as stack, file size, or address space are missing.

7. Add map shape coverage:

- Run ./map.
- Assert that it prints labels for stack, heap, globals, and code/text.
- Assert that address-looking values are present.
- Avoid hard-coding exact addresses because ASLR makes them unstable.

Suggested make test target

Add a test target that calls a script such as tests/run_hw_intro_tests.sh. That script should rebuild the homework, run the fixture tests, normalize output where needed, and return a non-zero exit status on the first failure.

HW 1 Lists

=====

Source file: hw-list/README.md

HW 1: Lists

=====

Official reference:

- <<https://cs162.org/static/hw/hw-list/>>
- <<https://cs162.org/static/hw/hw-list/docs/pwords/>>

Assignment goals

This homework introduces the Pintos intrusive linked-list implementation and uses it in a word-count program. It also introduces basic pthread programming: the pwords executable should spawn worker threads, process input files in parallel, and wait for all workers before printing the final result.

Key implementation areas

- list.c and list.h: Pintos list primitives and ownership conventions.
- word_count_l.c: list-backed word-count storage.
- word_count_p.c: thread-aware word-count storage and synchronization.
- pwords.c: create one thread per input file and join every spawned thread.
- pthread.c: smaller pthread practice program.
- gutenber/: larger input files for comparison and stress testing.

Build and run

```
cd hw-list
make
./pthread 4
./words gutenber/alice.txt
./lwords gutenber/alice.txt
./pwords gutenber/alice.txt gutenber/sawyer.txt
```

Manual comparison:

```
./words gutenber/alice.txt > /tmp/words.out
./lwords gutenber/alice.txt > /tmp/lwords.out
./pwords gutenber/alice.txt > /tmp/pwords.out
diff -u /tmp/words.out /tmp/lwords.out
diff -u /tmp/words.out /tmp/pwords.out
```

Current test inventory

No checked-in automated test harness was found for this homework. The current

manual checks compare the reference words program against lwords and pwords on larger text files.

Completeness assessment

Coverage is incomplete. The official homework centers on data-structure correctness and thread correctness, so tests should cover both deterministic word-count behavior and race-prone parallel behavior:

- lwords should match the reference output for all fixtures.
- pwords should match the reference output for one file and many files.
- The main pwords thread should always join workers before output.
- Shared word-count state should not lose increments under concurrent updates.
- List nodes should be inserted, searched, sorted, and freed without leaks.

Detailed missing-test prompts

1. Add deterministic fixtures under tests/fixtures/:

- empty.txt: empty file.
- single.txt: alpha beta alpha.
- case_punctuation.txt: Apple apple, APPLE!
- many.txt: repeat 100 to 1,000 words with known counts.

For every fixture, run:

```
""sh
./words tests/fixtures/single.txt > /tmp/ref.out
./lwords tests/fixtures/single.txt > /tmp/lwords.out
./pwords tests/fixtures/single.txt > /tmp/pwords.out
diff -u /tmp/ref.out /tmp/lwords.out
diff -u /tmp/ref.out /tmp/pwords.out
""
```

2. Add multi-file pwords coverage:

```
""sh
./words tests/fixtures/a.txt tests/fixtures/b.txt > /tmp/ref.out
./pwords tests/fixtures/a.txt tests/fixtures/b.txt > /tmp/pwords.out
diff -u /tmp/ref.out /tmp/pwords.out
""
```

The fixture should include the same word in both files so lost increments are visible.

3. Add a repeated race test:

```
""sh
for i in $(seq 1 100); do
./pwords tests/fixtures/many.txt tests/fixtures/many.txt > /tmp/run.out
diff -u /tmp/expected_twice.out /tmp/run.out
done
""
```

This catches missing locks, early printing before joins, and shared-list corruption that only appears intermittently.

4. Add argument and failure tests:

- ./pwords with no input should follow the documented behavior.
- ./pwords tests/fixtures/missing.txt should return non-zero and print a useful error.
- A mix of valid and invalid files should not leak threads or hang.

5. Add memory-check instructions:

```
“sh
valgrind --leak-check=full ./lwords tests/fixtures/many.txt
valgrind --leak-check=full ./pwords tests/fixtures/many.txt tests/fixtures/many.txt
”
```

These checks should confirm that list entries and word strings are released exactly once.

6. Add targeted list-unit tests if a small harness is acceptable:

- Insert several entries and confirm list order.
- Search for present and absent words.
- Increment an existing word without inserting a duplicate.
- Free a non-empty list and an empty list.

Suggested make test target

Add make test to rebuild the executables, run the fixture comparison script, and optionally run the repeated race loop when STRESS=1 is set.

HW 2 Shell

=====

Source file: hw-shell/README.md

HW 2: Shell

=====

Official reference:

- <<https://cs162.org/static/hw/hw-shell/>>
- <<https://cs162.org/static/hw/hw-shell/docs/directory-commands/>>
- <<https://cs162.org/static/hw/hw-shell/docs/signal-handling/task/>>

Assignment goals

This homework builds a small Unix-style shell. The official CS 162 task set covers command tokenization, directory commands, program execution, path resolution, redirection, pipes, and signal behavior.

The shell should behave like an interactive process manager: read commands, execute foreground jobs, preserve shell state across commands, and stay alive when user signals target a foreground child.

Key implementation areas

- shell.c: command loop, built-ins, fork/exec, waiting, redirection, pipes, and signal handling.
- tokenizer.c and tokenizer.h: parsing input into command tokens.
- test.sh: current smoke-test script.
- Makefile: build rules for the shell binary.

Build and run

```
cd hw-shell
make
bash test.sh
./shell
```

Manual command examples:

```
printf "pwd\n" | ./shell
printf "cd /npwd\n" | ./shell
printf "/bin/echo hello\n" | ./shell
printf "echo hello | wc -w\n" | ./shell
printf "echo saved > /tmp/hw-shell-out\n" | ./shell
```

Current test inventory

test.sh currently provides smoke coverage for:

- One built-in pwd command.
- One external /bin/lis command.

That is helpful, but it is not enough for the official shell behavior.

Completeness assessment

Coverage is incomplete. A shell is stateful and process-heavy, so tests should exercise sequences of commands, not only isolated one-line invocations.

Missing coverage includes:

- cd success and failure cases.
- pwd after directory changes.
- Empty input and whitespace-only input.
- External commands with arguments.
- Unknown commands and error handling.
- PATH lookup if the implementation supports it.
- Input and output redirection.
- Single and multi-stage pipes.
- Signal behavior for Ctrl-D, Ctrl-C, and Ctrl-Z.
- Tokenizer edge cases and malformed syntax.
- Child cleanup and non-hanging behavior.

Detailed missing-test prompts

1. Convert test.sh into a small assertion harness:

```
""sh
run_case() {
name="$1"
input="$2"
expected="$3"
actual="$(printf "%s" "$input" | ./shell)"
if [ "$actual" != "$expected" ]; then
printf "FAIL %s\nexpected:\n%s\nactual:\n%s\n" "$name" "$expected" "$actual"
exit 1
fi
}
""
```

Keep prompt filtering in one place if the shell prints prompts.

2. Add directory-command tests:

```
""sh
run_case "pwd root" "cd /npwd\n" "/"
run_case "cd relative" "cd /tmp\npwd\n" "/tmp"
""
```

Also add `cd /does/not/exist` and assert that the shell prints an error but continues to accept the next command.

3. Add external-command tests:

```
""sh
printf "/bin/echo hello cs162\n" | ./shell
printf "echo path lookup\n" | ./shell
printf "definitely-not-a-command\n/bin/echo survived\n" | ./shell
""
```

Assert that the invalid command does not terminate the shell.

4. Add redirection tests:

```
""sh
tmp="$(mktemp)"
printf "echo redirected > %s\n" "$tmp" | ./shell
grep -qx "redirected" "$tmp"
""
```

Add input-redirection coverage with `wc -w < fixture.txt`.

5. Add pipe tests:

```
""sh
printf "echo hello | wc -w\n" | ./shell
printf "echo alpha | tr a-z A-Z | wc -w\n" | ./shell
""
```

Assert the final output and ensure the shell exits cleanly after pipeline

children finish.

6. Add tokenizer unit tests:

Create a small tests/tokenizer_test.c that includes tokenizer.h and checks:

- Multiple spaces collapse correctly.
- Redirection symbols become separate tokens.
- Pipe symbols become separate tokens.
- Empty strings produce no command.
- Malformed commands return an error instead of reading invalid memory.

7. Add Ctrl-D behavior:

Start ./shell with no input from a here-document or closed stdin and assert that it exits normally rather than spinning.

8. Add Ctrl-C behavior:

Use a script that starts the shell, sends sleep 5, delivers SIGINT to the foreground process group, and then sends /bin/echo alive. The test passes only if sleep stops and the shell still runs the next command.

9. Add Ctrl-Z behavior:

Send SIGTSTP while a foreground command is running. The shell itself should not become stopped. Assert that a later command still executes.

Suggested make test target

Add a test target that rebuilds shell, runs the deterministic command tests, and only runs signal tests when the host supports Unix job-control semantics. Mark signal tests as Linux or WSL required.

HW 3 HTTP C

=====

Source file: hw-http/README.md

HW 3: HTTP Server in C

=====

Official reference:

- <<https://cs162.org/static/hw/hw-http/>>
- <<https://cs162.org/static/hw/hw-http/docs/tasks/socket/>>

Assignment goals

This homework implements an HTTP server in C. The official CS 162 task set covers socket setup, request parsing, GET handling, directory responses, proxying, multiple server variants, thread-pool support, and performance measurement.

The repository contains several server binaries so the same request-handling behavior can be validated under single-process, fork-per-connection, thread-per-connection, and thread-pool execution models.

Key implementation areas

- httpserver.c: server startup, request handling, file serving, proxy path, and concurrency-specific behavior.
- libhttp.c and libhttp.h: HTTP request and response helpers.
- wq.c and wq.h: work queue for the thread-pool server.
- www/: static file tree for manual testing.
- Makefile: builds httpserver, forkservice, threadserver, and poolserver.

Build and run

```
cd hw-http
make
./httpserver --files www --port 8000
```

From another terminal:

```
curl -i http://127.0.0.1:8000/
curl -i http://127.0.0.1:8000/my_documents/wholesome_facts.txt
curl -i http://127.0.0.1:8000/not-found
```

Repeat the same checks for the concurrency variants:

```
./forkserver --files www --port 8000
./threadserver --files www --port 8000
./poolserver --files www --port 8000 --num-threads 4
```

Current test inventory

No checked-in automated HTTP test harness was found for the C server. The current coverage is manual curl smoke testing only.

Completeness assessment

Coverage is incomplete. HTTP behavior should be verified over real sockets, because parser bugs, header formatting bugs, file I/O errors, and concurrency bugs often do not appear in compilation-only checks.

Missing coverage includes:

- Socket bind/listen/accept startup on all server variants.
- Valid GET for files and directories.
- Status codes for 200, 400, 403, 404, and 501 where applicable.

- Headers such as Content-Length, Content-Type, Connection, and response terminator formatting.
- MIME type selection.
- Directory listing behavior.
- Path traversal rejection.
- Malformed request handling.
- Proxy mode.
- Worker-queue behavior in poolserver.
- Concurrent clients and performance regression checks.

Detailed missing-test prompts

1. Add tests/http_smoke.sh:

- Build with make.
- Pick an unused port.
- Start a server in the background.
- Wait until the port accepts connections.
- Run curl checks.
- Kill the server in a trap on exit.

Suggested structure:

```
""sh
server="$1"
port="${PORT:-18080}"
"$server" --files www --port "$port" &
pid="$!"
trap 'kill "$pid" 2>/dev/null || true' EXIT
""
```

2. Add status-line tests for every server variant:

```
""sh
curl -sS -i "http://127.0.0.1:$port" | grep -q "HTTP/1.0 200"
curl -sS -i "http://127.0.0.1:$port/not-found" | grep -q "HTTP/1.0 404"
""
```

Run these against httpserver, forkserver, threadserver, and poolserver.

3. Add file-body and header tests:

- Request www/my_documents/wholesome_facts.txt.
- Assert the body contains a known line from the fixture.
- Assert Content-Length matches the fixture byte length.
- Assert Content-Type matches the extension.

4. Add directory tests:

- Request /.
- Assert the response is 200.
- Assert the body references expected entries from www/.
- Request a directory without a trailing slash if the assignment specifies a redirect or normalized response, and assert that behavior.

5. Add malformed request tests using nc or a small Python client:

```

""sh
printf "BADREQUEST\r\n\r\n" | nc 127.0.0.1 "$port"
printf "POST / HTTP/1.0\r\n\r\n" | nc 127.0.0.1 "$port"
""

```

Assert the documented error status and confirm the server keeps accepting later requests.

6. Add path traversal tests:

```

""sh
curl -sS -i "http://127.0.0.1:$port/../Makefile"
curl -sS -i "http://127.0.0.1:$port/%2e%2e/Makefile"
""

```

Assert that files outside www/ are not served.

7. Add concurrency tests:

```

""sh
seq 1 50 | xargs -n1 -P10 -l{} curl -fsS "http://127.0.0.1:$port/" >/dev/null
""

```

Run this for forserver, threadserver, and poolserver. The test should fail on any non-2xx response, crash, or hang.

8. Add wq unit tests:

Create tests/test_wq.c that links against wq.c and checks:

- Queue initializes empty.
- Push followed by pop returns the same pointer.
- Multiple items preserve FIFO order.
- Multiple producer and consumer threads do not lose items.
- Shutdown or destroy behavior does not leave waiting threads stuck.

9. Add proxy tests:

- Start a local backend with python3 -m http.server 19090.
- Start the homework server in proxy mode.
- Request through the homework server.
- Assert that backend content is returned and that backend failure produces a documented error response.

Suggested make test target

Add a test target that runs tests/http_smoke.sh for each binary. Keep a separate make perf or make stress target for high-concurrency and benchmark runs so the default test remains fast.

HW 3 HTTP Rust

=====

Source file: hw-http-rs/README.md

HW 3: HTTP Server in Rust

=====

Official reference:

- <<https://cs162.org/static/hw/hw-http-rs/>>
- <<https://cs162.org/static/hw/hw-http-rs/docs/requests/>>

Assignment goals

This homework implements the CS 162 HTTP server in Rust. It mirrors the C HTTP homework while using Rust libraries and async I/O. The official task set covers argument parsing, socket handling, request parsing, GET responses, directory handling, response-code statistics, and performance-oriented testing.

Key implementation areas

- src/main.rs: binary entry point and server startup.
- src/args.rs: command-line argument parsing.
- src/http.rs: response formatting, headers, status lines, and MIME helpers.
- src/server.rs: socket accept loop and request handling.
- src/stats.rs: shared response-code counters.
- src/tests/: current helper-level Rust tests.
- www/: static file tree for manual server testing.

Build and run

```
cd hw-http-rs
cargo build
cargo test
cargo run -- --files www --port 8000
```

From another terminal:

```
curl -i http://127.0.0.1:8000/
curl -i http://127.0.0.1:8000/my_documents/wholesome_facts.txt
curl -i http://127.0.0.1:8000/not-found
```

Current test inventory

The repository currently has helper-level Rust tests in src/tests/:

- args.rs: default argument behavior and manual construction.
- http.rs: MIME type helpers and response helper formatting.
- stats.rs: counter initialization, increments, sorted output, debug formatting, and shared stats updates.

These tests are useful, but they do not start the server or send real HTTP requests.

Completeness assessment

Coverage is partial. The helper tests protect small functions, but the main assignment behavior lives at the socket and request-handler boundary.

Missing coverage includes:

- parse_request behavior for valid, invalid, partial, and unsupported requests.
- Real file serving over TCP.
- Directory responses.
- Missing-file and forbidden-path behavior.
- Header and body correctness for real responses.
- Response-code statistics after real requests.
- Concurrent clients.
- Graceful handling of malformed clients that disconnect early.

Detailed missing-test prompts

1. Add request-parser unit tests:

In src/tests/http.rs or a new src/tests/requests.rs, create cases for:

- GET /index.html HTTP/1.0\r\n\r\n
- GET / HTTP/1.1\r\nHost: localhost\r\n\r\n
- POST / HTTP/1.0\r\n\r\n
- Empty input.
- A request line with too few fields.
- A path containing ...

Assert the exact parsed method, path, and error type expected by the implementation.

2. Add a Tokio integration test under tests/server_integration.rs:

- Create a temporary directory.
- Write index.html and another small file.
- Bind the server to port 0 or otherwise allocate a free port.
- Connect with tokio::net::TcpStream.
- Send a raw HTTP request.
- Read the full response and assert status line, headers, and body.

3. Add real GET response checks:

```
""text
GET /index.html HTTP/1.0\r\n\r\n
""
```

Assert:

- Status contains 200.
- Content-Length equals the fixture byte length.
- Content-Type matches the extension.
- Body equals the file contents.

4. Add missing-file and traversal checks:


```
""text
GET /missing.txt HTTP/1.0\r\n\r\n
GET ../Cargo.toml HTTP/1.0\r\n\r\n
""
```

Assert that missing files return the documented not-found status and that traversal attempts do not expose files outside the configured root.

5. Add directory checks:

- Request /.
- Assert either the configured index file or directory listing behavior, depending on the implementation path required by the assignment.
- Request a nested directory fixture and assert stable output.

6. Add stats integration checks:

- Send two successful requests and one missing-file request.
- Query or inspect the shared Stats object if the server exposes it.
- Assert the 200 and 404 counters changed by the expected amounts.

7. Add concurrent-client coverage:

```
""rust
let requests = (0..50).map(|_| tokio::spawn(send_get(port, "/index.html")));
""
```

Await all tasks and assert every response is successful. This catches accidental shared-state locks held too long and request-handler panics.

8. Add disconnect and partial-request coverage:

- Open a TCP connection and close it without sending a full request.
- Send GET / HTTP/1.0\r\n without the terminating blank line.
- Assert the server does not panic and still accepts a later valid request.

Suggested commands

```
cargo test
cargo test --test server_integration
cargo test -- --nocapture
```

Keep fast parser/helper tests in the default suite, and keep slower concurrent server tests in integration tests so they can be isolated when debugging.

HW 4 Memory

=====

Source file: hw-memory/README.md

HW 4: Memory

=====

Official reference:

- <<https://cs162.org/static/hw/hw-memory/>>
- <<https://cs162.org/static/hw/hw-memory/docs/sbrk/introduction/>>
- <<https://cs162.org/static/hw/hw-memory/docs/library-functions/introduction/>>

Assignment goals

This homework focuses on user memory management. The official CS 162 memory homework asks students to add sbrk-style heap growth to Pintos and implement user-level allocation functions such as malloc, calloc, realloc, and free. The repository also contains a standalone allocator practice directory, mm_alloc/, which should be tested separately from the Pintos suite.

Key implementation areas

-
- mm_alloc/mm_alloc.c: standalone mm_malloc, mm_realloc, and mm_free implementation.
 - mm_alloc/mm_test.c: current smoke test for the standalone allocator.
 - pintos/src/userprog/ and related Pintos code: syscall and process memory behavior used by sbrk.
 - pintos/src/lib/user/: user-level allocation library functions.
 - pintos/src/tests/memory/: Pintos memory tests and expected-output checks.
 - pintos/src/tests/memory/Make.tests: active Pintos memory test list.

Build and run

Standalone allocator:

```
cd hw-memory/mm_alloc
make
./mm_test
```

Pintos memory suite in the CS 162 Linux environment:

```
cd hw-memory/pintos/src/memory
make
make check
```

Run one Pintos memory test after the build directory exists:

```
cd hw-memory/pintos/src/memory/build
make tests/memory/malloc-simple.result
```

Current test inventory

Coverage is mixed:

- mm_alloc/mm_test.c is a minimal smoke test for the standalone allocator.
- pintos/src/tests/memory/Make.tests enables 26 active Pintos memory tests.
- Active Pintos tests cover sbrk, heap growth, malloc, free, realloc, alignment, out-of-memory behavior, and several stress cases. calloc is part of the official allocator surface and should receive explicit extra tests.
- Stack-growth .ck files are present, but stack-growth test entries are commented out in Make.tests.

Completeness assessment

The Pintos memory test suite is the strongest coverage in the repository, but the standalone allocator is under-tested. Also, Pintos tests must be run in the course environment because they depend on the Pintos build system, emulator, and expected 32-bit toolchain behavior.

Missing standalone allocator coverage includes:

- 8-byte alignment.
- Zero-size allocation policy.
- calloc zero initialization if calloc is part of the local API.
- realloc(NULL, size) and realloc(ptr, 0) behavior.
- Realloc growth and shrink copy preservation.
- Free-list reuse after free.
- Coalescing adjacent free blocks.
- Fragmentation stress.
- Invalid or repeated free behavior, if the assignment defines expectations.

Detailed missing-test prompts

1. Expand mm_alloc/mm_test.c or add mm_alloc/tests/mm_alloc_cases.c:

- Allocate 1, 2, 7, 8, 9, 16, 31, 64, 256, and 4096 bytes.
- Assert every returned pointer is non-null when enough heap exists.
- Assert (uintptr_t)ptr % 8 == 0 for every allocation.
- Fill every allocation with a byte pattern and verify no adjacent allocation is corrupted.

2. Add reuse-after-free coverage:

```

""c
void *a = mm_malloc(64);
void *b = mm_malloc(64);
mm_free(a);
void *c = mm_malloc(32);
""

```

Assert c is valid and that writing to c does not corrupt b. If the allocator is expected to reuse free blocks, assert that behavior explicitly.

3. Add realloc growth coverage:

- Allocate 32 bytes.
- Fill bytes 0..31.
- Reallocate to 128 bytes.
- Assert the first 32 bytes are preserved.
- Write the new region to confirm it is usable.

4. Add realloc shrink coverage:

- Allocate 128 bytes.
- Fill a known pattern.
- Reallocate to 16 bytes.
- Assert the first 16 bytes are preserved.
- Free the result.

5. Add realloc special-case coverage:

- `mm_realloc(NULL, 64)` should behave like allocation if the API follows C `realloc`.
- `mm_realloc(ptr, 0)` should follow the documented policy, usually freeing `ptr` and returning `NULL`.
- Document the expected behavior if this assignment uses a different policy.

6. Add fragmentation coverage:

- Allocate 100 small blocks.
- Free every other block.
- Allocate medium-size blocks.
- Assert the allocator does not fail prematurely.
- Free everything and repeat.

7. Add stress coverage:

- Generate a deterministic sequence of allocate, free, and realloc operations.
- Keep a shadow table of live blocks and expected byte patterns.
- Verify all live blocks after every 50 operations.

8. Add sanitizer or Valgrind instructions:

```
""sh
make clean
CFLAGS="-g -O0 -fsanitize=address,undefined" make
./mm_test
valgrind --leak-check=full ./mm_test
""
```

Use whichever tool is available in the Linux environment.

9. Add a new Pintos memory test only when a behavior is not already covered:

- Create `pintos/src/tests/memory/<name>.c`.
- Create matching `pintos/src/tests/memory/<name>.ck`.
- Add the test name to the appropriate list in `Make.tests`.
- Run:

```
""sh
cd pintos/src/memory
make check
cd build
make tests/memory/<name>.result
""
```

10. Treat disabled stack-growth tests carefully:

If stack growth is required for the current assignment version, uncomment the stack-growth entries in Make.tests and record the reason. If it is not required, leave them disabled and state that decision in the submission notes.

Suggested testing standard

Before considering this homework well tested, collect both signals:

- mm_alloc passes deterministic allocator unit tests and a stress run.
- Pintos make check passes the active memory tests in the course Linux environment.

HW 5 MapReduce C

Source file: hw-map-reduce/README.md

HW 5: MapReduce in C

Official reference:

- <<https://cs162.org/static/hw/hw-map-reduce/>>
- <<https://cs162.org/static/hw/hw-map-reduce/docs/job-submission/>>

Assignment goals

This homework implements a distributed MapReduce system in C. The official CS 162 task set covers job submission, worker registration, task distribution, worker heartbeats, task completion, fault tolerance, and output processing.

The coordinator should maintain durable job and task state, workers should request and execute tasks, and the client should submit jobs and process final outputs from sample applications.

Key implementation areas

- rpc/rpc.x: RPC interface shared by coordinator, worker, and client.
- coordinator/coordinator.c: RPC service entry points and scheduler logic.
- coordinator/job.c: job metadata, task state, queues, and transitions.
- worker/worker.c: worker registration, polling, heartbeat, and task loop.
- worker/task_handler.c: map and reduce task execution.
- client/client.c: job submission, polling, and output processing.
- app/wc/, app/grep/, app/vertex_degree/: sample MapReduce apps.
- data/: sample input data.

Build and run

```
cd hw-map-reduce
make
```

Manual three-terminal smoke test:

Terminal 1:

```
./bin/mr-coordinator
```

Terminal 2:

```
./bin/mr-worker
```

Terminal 3:

```
rm -rf /tmp/hw-map-reduce-out
mkdir -p /tmp/hw-map-reduce-out
./bin/mr-client submit -a wc -o /tmp/hw-map-reduce-out -w data/gutenberg/p.txt
./bin/mr-client process -a wc -o /tmp/hw-map-reduce-out
```

Current test inventory

No checked-in automated harness was found for the C MapReduce implementation.
The current coverage is manual cluster execution only.

Completeness assessment

Coverage is incomplete. MapReduce correctness depends on multi-process coordination, so the important tests must start real coordinator and worker processes, submit real jobs, and compare output against golden files.

Missing coverage includes:

- Job IDs are unique and start from the expected value.
- Job submission deeply copies input paths, output paths, app names, and app arguments.
- Output directories are created when missing.
- Map and reduce tasks are initialized as not started.
- Workers receive unique IDs.
- Multiple workers share tasks without duplicate ownership.
- Completed tasks advance job state correctly.
- Worker failure causes task retry.
- Sample apps produce deterministic output.
- Invalid job submissions fail cleanly.

Detailed missing-test prompts

1. Add tests/run_cluster_test.sh:

- Build with make.

- Create a temporary output directory.
- Start ./bin/mr-coordinator in the background.
- Start one or more ./bin/mr-worker processes.
- Submit a job.
- Poll until completion.
- Run mr-client process.
- Compare processed output against a golden file.
- Clean up all background processes in a trap.

Process cleanup is essential because failed MapReduce tests otherwise leave coordinators or workers running on the host.

2. Add word-count golden coverage:

Create tests/fixtures/wc_input.txt:

```
""text
alpha beta alpha
beta gamma
""
```

Expected processed output should include:

```
""text
alpha 2
beta 2
gamma 1
""
```

Sort output before comparison if reducer output order is not guaranteed.

3. Add multi-file job coverage:

Submit one job with two or more -w input files. Put overlapping words in multiple files so the reduce step must combine mapper outputs correctly.

4. Add multiple-worker coverage:

Start three workers and submit a job with enough map tasks to distribute. Assert the job completes and output matches the single-worker golden result. This catches coordinator state bugs that only appear with concurrent worker requests.

5. Add worker-failure coverage:

- Start the coordinator and two workers.
- Submit a job with multiple tasks.
- Kill one worker while the job is running.
- Assert the coordinator marks its in-progress task available again.
- Assert the final output still matches the golden result.

6. Add invalid submission tests:

- Unknown app name.
- Missing input file.
- Output path that cannot be created.
- Invalid reducer count if the client exposes that option.

Each should return a failure status without crashing the coordinator.

7. Add coordinator-state unit tests if internal functions are linkable:

- Submitting the first job returns ID 0.
- Submitting a second job returns ID 1.
- Newly submitted jobs have all map and reduce tasks in NOT_STARTED.
- Deep-copied strings remain valid after the request object is freed.
- Output directory creation is attempted exactly once.

8. Add sample-app coverage:

- wc: compare against a small word-count golden file.
- grep: use a fixture where only known lines match.
- vertex_degree: use a tiny graph with known degrees.

9. Add timeout protection:

Every integration test should fail after a fixed timeout, for example 30 seconds, so scheduler deadlocks become visible instead of hanging forever.

Suggested make test target

Add make test to run the fast deterministic integration cases. Keep worker-failure and large stress tests behind a separate make stress target if they are slower or more environment-sensitive.

HW 5 MapReduce Rust

=====

Source file: hw-map-reduce-rs/README.md

HW 5: MapReduce in Rust

=====

Official reference:

- <<https://cs162.org/static/hw/hw-map-reduce-rs/>>
- <<https://cs162.org/static/hw/hw-map-reduce-rs/docs/worker-registration/>>

Assignment goals

This homework implements the CS 162 MapReduce system in Rust. It uses async Rust, generated protobuf types, a coordinator service, worker services, and a client. The official task set covers worker registration, job submission, task distribution, task completion, heartbeats, retries, and fault tolerance.

Key implementation areas

- proto/: coordinator and worker protobuf definitions.
- src/coordinator/: coordinator state, RPC methods, task queues, and retry behavior.
- src/worker/: worker registration, heartbeat, task execution, and reporting.
- src/bin/: mr-coordinator, mr-worker, mr-client, and

mr-autograder.

- src/tests/mod.rs: current draft integration test module.
- data/: sample inputs.

Build and run

```
cd hw-map-reduce-rs
cargo build
cargo test
```

Run the ignored draft word-count test after replacing its placeholder input:

```
cargo test test_wc -- --ignored
```

Manual service run in separate terminals:

```
cargo run --bin mr-coordinator
cargo run --bin mr-worker
cargo run --bin mr-client -- submit -a wc -o /tmp/hw-map-reduce-rs-out -w
data/gutenberg/p.txt
cargo run --bin mr-client -- process -a wc -o /tmp/hw-map-reduce-rs-out
```

Current test inventory

The repository currently has an ignored test_wc in src/tests/mod.rs, but the test still contains a placeholder input path. As a result, default cargo test does not validate a successful MapReduce job.

Completeness assessment

Coverage is incomplete. The Rust implementation needs both coordinator-state unit tests and async integration tests. The official assignment relies heavily on distributed state transitions, so the test suite should make those transitions explicit.

Missing coverage includes:

- Worker IDs are unique and start from the expected value.
- Worker registration stores worker metadata correctly.
- Job IDs are unique.
- New jobs create all map and reduce tasks in the right initial state.
- Task assignment respects map-before-reduce ordering.
- Task completion updates job state.
- Duplicate completion reports do not corrupt state.
- Worker heartbeat timeout triggers retry.
- End-to-end wc, grep, and vertex_degree outputs match golden files.
- Failed workers and cancelled tasks do not hang the cluster.

Detailed missing-test prompts

1. Finish src/tests/mod.rs::test_wc:

- Replace todo!("Pick an input file") with a checked-in fixture or a temporary file created inside the test.
- Submit a wc job through the client path.
- Wait for completion with a timeout.
- Process output.
- Sort or normalize output if reducer order is not deterministic.
- Assert the expected counts.

Example fixture:

```
""text
alpha beta alpha
beta gamma
""
```

Expected output: alpha 2, beta 2, gamma 1.

2. Add worker-registration unit tests:

- Call the coordinator registration logic twice.
- Assert returned worker IDs are 0 and 1 if IDs start from zero.
- Assert both workers are present in coordinator state.
- Assert worker address or heartbeat metadata is stored.

3. Add job-submission unit tests:

- Submit two jobs and assert job IDs are sequential.
- Assert input file lists, output directory, app name, app args, and reducer count are copied into coordinator state.
- Assert every map task starts as not started.
- Assert reduce tasks are not assigned before map tasks finish.

4. Add task-assignment tests:

- Register two workers.
- Submit a job with multiple input files.
- Request tasks from both workers.
- Assert they do not receive the same in-progress task unless retry logic has intentionally made the task available again.

5. Add task-completion tests:

- Mark a map task complete and assert its state changes.
- Complete all map tasks and assert reduce tasks become assignable.
- Complete all reduce tasks and assert the job reaches done state.
- Send a duplicate completion report and assert it is ignored safely.

6. Add heartbeat and retry tests:

- Register a worker and assign it a task.
- Advance the coordinator's notion of time or wait past the heartbeat timeout.
- Assert the task returns to the available queue.
- Assert another worker can receive and finish that task.

7. Add integration tests using the existing cluster utilities:

- Start an in-process coordinator and multiple workers, for example with `utils::start_cluster` if that helper is available.
- Use `#[tokio::test]`.
- Use `tokio::time::timeout` around the whole test.
- Submit `wc`, `grep`, and `vertex_degree` jobs using tiny deterministic fixtures.

8. Add invalid-input integration tests:

- Unknown app name.
- Missing input path.
- Output directory that cannot be created.
- Zero reducers if the API rejects that case.

Assert the client receives a clean error and the coordinator remains alive.

9. Add output-cleanup checks:

- Use `tempfile::TempDir` for each test.
- Assert intermediate files are written where expected.
- Assert processed output is deterministic.
- Let `TempDir` remove generated files after the test.

Suggested commands

```
cargo test
cargo test test_wc -- --ignored
cargo test -- --nocapture
```

Keep deterministic state-machine tests active in the default cargo test suite. Keep slow fault-tolerance tests active if they are reliable; otherwise mark them ignored with a clear comment and run them in CI or before submission.

Test Completeness Audit

=====

The standalone audit is included as a separate document and PDF:

- `test-completeness-audit.md`
- `test-completeness-audit.pdf`